

Implementation of a Dual-Filtered CNN-Based Hand Gesture Recognition System

Danyal Ahmad

Department of Computer Science
Vishwakarma Institute Of Information Technology
danyalahmad.22311938@viit.ac.in

Shivam Koli

Department of Computer Science
Vishwakarma Institute Of Information Technology
shivam.22311902@viit.ac.in

Malhar Khandekar

Department of Computer Science
Vishwakarma Institute Of Information Technology
malhar.22311890@viit.ac.in

Pathan Arsh

Department of Computer Science
Vishwakarma Institute Of Information Technology
arsh.22311910@viit.ac.in

Prof.Yogesh Sharma

Department of Computer Science
Vishwakarma Institute Of Information Technology
yogesh.sharma@viit.ac.in

Prof.Rahul Diwate

Department of Computer Science
Vishwakarma Institute Of Information Technology
rahul.diwate@viit.ac.in

Abstract—This paper presents the practical implementation of a real-time hand gesture recognition system based on a dual-filtered convolutional neural network (CNN). The system interprets static sign language alphabets (A–Z, excluding J and Z) using two trained CNN models: one based on finger segmentation and another on silhouette edge detection. A hybrid fusion technique combines the output probabilities of both models to ensure robust classification under varying lighting and environmental conditions. Implemented using Python, OpenCV, Mediapipe, and PyTorch, the system achieves accurate results in real-time, demonstrating its viability for gesture-based human–computer interaction.

I. INTRODUCTION

Sign language recognition has gained significant attention as an assistive technology for bridging communication gaps between hearing-impaired and non-signers. While previous research has focused on theoretical model comparisons, this paper details a full-scale implementation of a dual-filter CNN system capable of real-time operation. The project focuses on preprocessing, model integration, and environmental testing to evaluate performance robustness.

II. HARDWARE SETUP

The hardware configuration for the system is minimal and designed for flexibility. A standard RGB camera is positioned in front of the user at approximately chest or shoulder height. Lighting is preferred from the front, at approximately 60° relative to the camera's optical axis. Direct backlighting (light behind the hand) is avoided as it causes silhouette loss and misclassification. Backgrounds can be cluttered or noisy since

the filtering pipeline isolates the hand effectively. A decent quality camera (360p or above) suffices for real-time inference.

III. SYSTEM ARCHITECTURE

The proposed system is divided into five major modules, integrated sequentially to form a continuous processing pipeline.

A. Camera Capture Module

Continuously captures frames using OpenCV. Each frame is resized and sent to the Mediapipe detector for hand landmark identification.

B. Hand Detection and Landmark Module

Uses Mediapipe's Hands solution to locate 21 key points of the hand in real time. The landmarks are stored and passed to the filtering module.

C. Filtering Module

Two preprocessing techniques are used simultaneously:

- **Edge Detection Filter** – Generates hand silhouettes emphasizing outer contours.
- **Finger Color Segmentation Filter** – Distinguishes individual fingers using Mediapipe landmarks and assigns distinct color codes for each finger (thumb, index, middle, ring, pinky) and palm.

D. CNN Prediction Module

Both CNN models (Fingers-CNN and Silhouette-CNN) independently predict gesture probabilities. A weighted fusion mechanism combines their predictions:

$$P_{final} = 0.6 \times P_{fingers} + 0.4 \times P_{silhouette}$$

The final class label is determined by the maximum probability score.

E. Output and GUI Module

The final recognized alphabet is displayed on-screen through a graphical interface built using Streamlit, allowing real-time visual feedback.

The finger detection filter colors individual fingers for neural networks to learn easily, this also avoids overlap confusion which was the drawback of Binary filtering.

IV. CODE SNIPPET: FILTERING IMPLEMENTATION

The following code demonstrates the key preprocessing steps used to generate binary and finger-colored masks for CNN input.

```
1 # filters.py
2 import cv2
3 import numpy as np
4
5 def build_mask_from_landmarks(img,
6     hand_landmarks_list, mp_hands, mp_drawing):
7     h, w, _ = img.shape
8     mask_color = np.zeros((h, w, 3), dtype=np.uint8)
9     for hand_landmarks in hand_landmarks_list:
10         mp_drawing.draw_landmarks(
11             mask_color, hand_landmarks, mp_hands.
12             HAND_CONNECTIONS,
13             mp_drawing.DrawingSpec(color=(255, 255,
14             255), thickness=3, circle_radius=2)
15             )
16     mask_gray = cv2.cvtColor(mask_color, cv2.
17     COLOR_BGR2GRAY)
18     mask_gray = cv2.dilate(mask_gray, np.ones((5, 5),
19     np.uint8), iterations=2)
20     mask_gray = cv2.morphologyEx(mask_gray, cv2.
21     MORPH_CLOSE, np.ones((7, 7), np.uint8),
22     iterations=1)
23     return mask_gray
```

Listing 1: Filtering Implementation in Python

```
1 def preprocess_to_silhouette_and_edges(frame_bgr,
2     hand_mask):
3     hand_only = cv2.bitwise_and(frame_bgr, frame_bgr
4     , mask=hand_mask)
5     gray = cv2.cvtColor(hand_only, cv2.
6     COLOR_BGR2GRAY)
7     blur = cv2.medianBlur(gray, 5)
8     _, silhouette = cv2.threshold(blur, 0, 255, cv2.
9     THRESH_BINARY + cv2.THRESH_OTSU)
10    silhouette = cv2.morphologyEx(silhouette, cv2.
11    MORPH_CLOSE, np.ones((3, 3), np.uint8),
12    iterations=2)
13    silhouette = cv2.morphologyEx(silhouette, cv2.
14    MORPH_OPEN, np.ones((3, 3), np.uint8),
15    iterations=1)
16    edges = cv2.Canny(silhouette, 3, 100)
17    edges = cv2.dilate(edges, np.ones((3, 3), np.
18    uint8), iterations=1)
19    return silhouette, edges
```

Listing 2: Silhouette and Edge Preprocessing

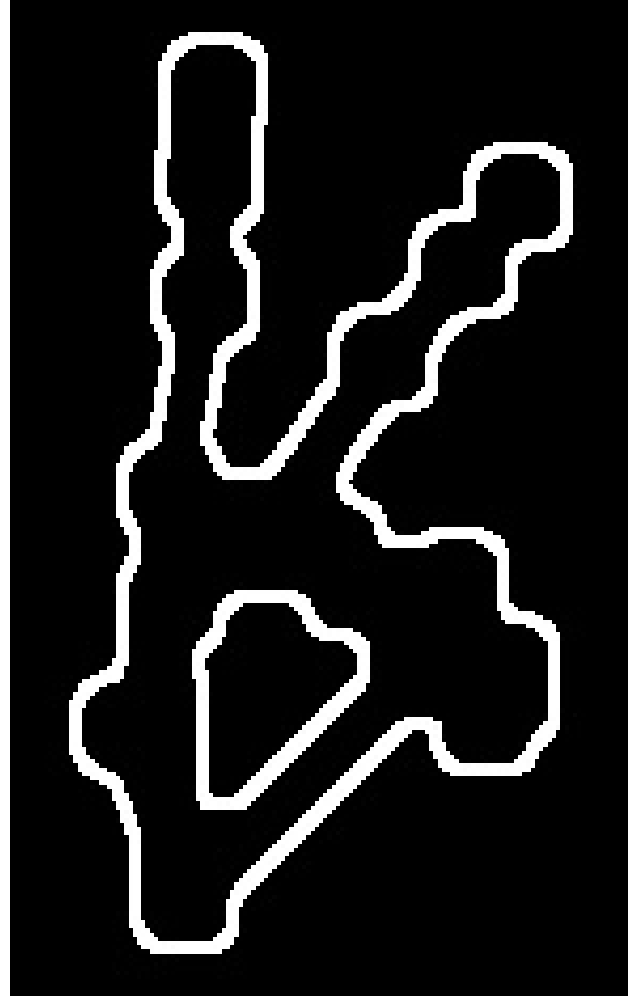


Fig. 1: Binary filtering for simplifying image.

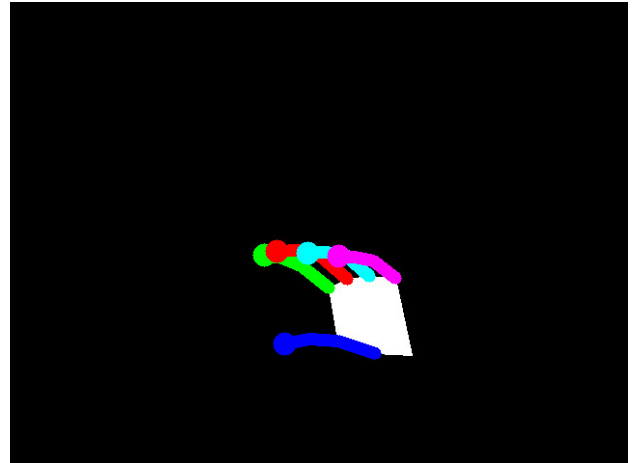


Fig. 2: Each Finger Detection filter.

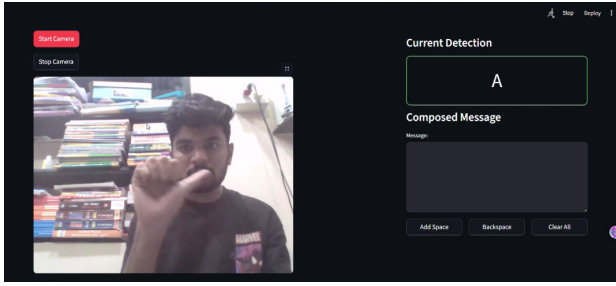


Fig. 3: GUI for Handsign translation

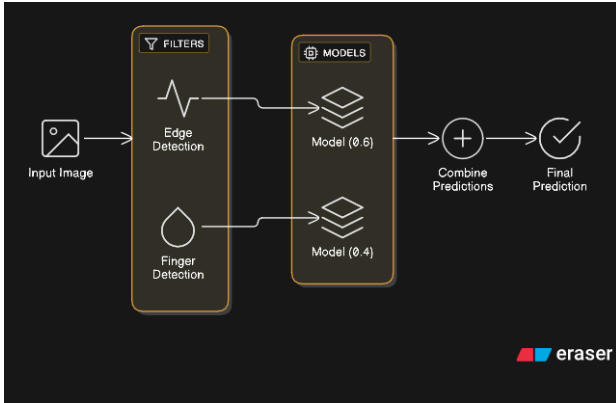


Fig. 4: Proposed System Architecture

V. WORKFLOW DIAGRAM

The overall workflow (to be inserted diagrammatically):

Camera → Mediapipe Hand Detection → Dual Filtering (Edge + Finger) → CNN Predictions → Weighted Fusion → Output Display

VI. TESTING AND VALIDATION

Extensive real-world testing was performed under different lighting and background conditions.

TABLE I: Environmental Testing Results

Lighting Condition	Description	Result
Front light (60°)	Light in front of camera	High accuracy
Side lighting	Light from left/right	Slightly reduced
Back lighting	Light behind hand	Poor output
Dim / No light	Low visibility	Undetected
Background clutter	Objects behind hand	No effect

The filtering pipeline effectively nullified background influence, while lighting angle remained the key factor affecting accuracy.

VII. RESULTS

The models were trained on 120,000 total images for 24 alphabetic gestures. Average training time: 1.5 hours on NVIDIA GTX 1060 GPU.

The final implementation achieved near real-time prediction capability with high consistency.

TABLE II: Model Performance Summary

Model	Train Acc.	Test Acc.	Role
CNN – Fingers	98%	94%	Primary model
CNN – Silhouette	96%	92%	Secondary verifier
Combined (Fusion)	–	95.5%	Final output

VIII. DISCUSSION AND LIMITATIONS

The system performs efficiently in real-time scenarios with minimal latency. However, several limitations were identified:

- **Heavy training load:** Approximately 1.5 hours for 120k images on a mid-range GPU (GTX 1060). Scaling up increases computation time significantly.
- **Lighting dependency:** Poor backlighting affects silhouette visibility.
- **Integration challenges:** Minor accuracy drop observed during GUI-model integration due to frame delay.

Despite these challenges, the dual-filter CNN architecture proved robust, reducing background effects and maintaining gesture distinction across subjects.

IX. CONCLUSION

The implemented dual-filter CNN system successfully recognized static hand gestures from A–Z (excluding J and Z) in real-time using only a standard camera. The combination of binary edge filtering, Mediapipe-based finger segmentation, and probability fusion enhanced accuracy and reliability. The system demonstrates the feasibility of deploying lightweight deep learning models for sign language recognition on consumer-grade hardware, suitable for assistive communication and gesture-based control applications.

REFERENCES

- [1] “Real-Time Sign Language Detection using TensorFlow, OpenCV and Python,” IJRASET. Available: <https://www.ijraset.com/research-paper/real-time-sign-language-detection-using-tensorflow-opencv-and-python>
- [2] “Sign Language Recognition,” ResearchGate. Available: https://www.researchgate.net/publication/353141966_Sign_Language_Recognition.
- [3] “Sign Language Recognition,” IOP Conference Series: Materials Science and Engineering, vol. 333, 2018. Available: <https://iopscience.iop.org/article/10.1088/1757-899X/333/1/012109>
- [4] Shirin Tikoo, “Develop Sign Language Translator with Python — Skyfi Labs,” Skyfi Labs, Published: 2019-11-26; Last Up- dated: 2022-05-14. Available: <https://www.skyfilabs.com/project-ideas/sign-language-translator-using-python>
- [5] Ramswarup Kulhary, “OpenCV - Python,” GeeksforGeeks, up- dated 05 August 2021. Available: <https://www.geeksforgeeks.org/opencv-overview/>
- [6] IOP Conference Series: Journal article, vol. 574, 2014. Available: <https://iopscience.iop.org/article/10.1088/1742-6596/574/1/012109>

[7] IOP Conference Series: Journal article.
Available: <https://iopscience.iop.org/article/10.1088/1742-6596/490/1/012103>

[8] S. Sharma et al., “A Real-Time Indian Sign Language Recognition System Using CNNs.”

[9] H. Zhang and K. Wang, “Deep Learning Based Gesture Recognition for Accessibility.”

[10] P. Mehta, “Computer Vision for Human Gesture Analysis,” *Proc. IEEE Conf. CVPR*, 2019.

[11] ‘Image Classification using Convolutional Neural Networks’ by Muthukrishnan Ramprasath, Shanmugasundaram Hariharan and Ram Prasath.

[12] ‘An Analysis Of Convolutional Neural Networks For Image Classification’ by Neha Sharma.

[13] ‘Convolutional Neural Network (CNN) for Image Detection and Recognition’ by Rahul Chauhan, Kamal Kumar Ghanshala and R. C. Joshi.